

Project: CNCF · urunc · Improve lifecycle management of sandbox monitors (urunc-dev/urunc#112)

Mentors: @cmainas,
@ananos

1. About me

I am Raman Kapoor, a B.Tech (Information Technology) graduate from IIIT Gwalior (class of 2026), currently working at Juspay, India's largest payments orchestration platform, on low-level Linux infrastructure. My most recent landed piece there was leading the migration of our block-storage tier from CEPH to Lightbits NVMe/TCP — same workload at 48K TPS versus 37K TPS on roughly a tenth of the hardware (four nodes instead of forty). Before that I shipped a thread-local and server-local cache layer in Rust on our hot APIs that cut cross-node calls by ~4% at peak, and a Go-based Shamir-secrets distribution broker on top of OpenBao that removed our reliance on AWS KMS for key delivery. Right now I am on the bare-metal datacenter side of the stack, writing the secure-erase command for our internal Rust CLI that drives MAAS-managed hosts through their lifecycle.

2. What I have been working on

2.1 Open-source contributions to Namma Yatri (Haskell, production)

The work I am proudest of in a large unfamiliar codebase with maintainer review is my contribution to Namma Yatri, the open-source ride-hailing platform that runs in production across India. Between November 2025 and January 2026 I shipped three merged PRs into the org:

- **feat** [nammayatri/nammayatri#13442](#) — Special-zone driver-payout scheduling system. A distributed, event-driven scheduler serving roughly 10,000 rides per day; cut driver payout latency from weekly cycles down to T+2 hours under production traffic.
- **feat** [nammayatri/nammayatri#13452](#) — Exposed the resulting driver-payout state to the driver-app frontend through the API layer.
- **feat** [nammayatri/shared-kernel#1108](#) — PaytmEDC payment-flow integration in the shared kernel: config management, checksum generation, API interaction. Went through several review iterations before merge.

The technically interesting one is #13442. Driver payouts for special-zone rides (airports, stations) are computed per-ride but were previously paid out only on a weekly batch cycle — a long latency for drivers, and operationally fragile when the batch failed. The new system schedules per-ride payouts at T+2 hours after ride completion, which means roughly 10,000 scheduled jobs in flight at any given moment under load. The constraints made the design non-trivial:

- **Exactly-once execution.** A payout that fires twice is real money lost; a payout that never fires is a real driver not paid. The scheduler had to be exactly-once under broker restart, app restart, and partial-failure scenarios.

- **No latency on the ride-booking hot path.** Scheduling a payout is a side-effect of completing a ride; the ride-completion API could not block on scheduling state.
- **Idempotent state transitions.** A retry of the payout-fire call had to be safe; the same input must yield the same end state, not double-debit.

The shape I landed on is a Redis-backed delayed-job scheduler keyed on ride-id, with per-ride distributed locks taken at fire time and an idempotency token persisted before the actual payout call. The per-ride lock ensures only one worker fires a given payout even if two pick it up simultaneously, and the idempotency token means a retry after a partial failure is safe. Working on a Haskell codebase with no prior production Haskell experience, designing the failure model with the team, and getting to merge three times in a row, is the closest thing I have done to what this LFX project will involve — and it is the muscle I am leaning on for that reason.

2.2 Juspay — production infrastructure work

2.2.1 Identity and secrets on OpenStack + OpenBao

We replaced a heavy AWS KMS dependency with an in-house identity-and-secrets layer running on OpenStack VMs and OpenBao. A few design choices shaped most of what I built:

- **Identity derived from the network, not from a long-lived token.** A VM authenticating to the broker presents an IMDS-attested claim — its identity is the network position the orchestrator placed it at, signed by the platform. There is no static credential on disk an attacker can steal. The broker validates the attestation and issues a short-lived RS256-signed token scoped to what that VM is allowed to read.
- **Shamir secret-sharing for the root key.** No single broker node holds the full root; it is split into m -of- n shards distributed across the cluster, and reassembled in memory only when needed. A single compromised node yields no useful secret material.
- **HA via Raft.** Brokers run a Raft cluster across availability zones, so the identity service tolerates $N-1$ broker failures without an outage — the same posture etcd or CockroachDB take, applied to secrets.
- **VM-side SDK in Python (`osvault`)** for the bootstrap flow — handles attestation, token refresh, leasing, and audit. The broker itself is Go.

2.2.2 Bare-metal datacenter lifecycle

My current piece of work at Juspay is on `namma-cli`, a Rust CLI that drives the lifecycle of bare-metal hosts in our datacenter, sitting on top of MAAS for hardware control. Hosts are described declaratively in site specs; `namma-cli` reconciles spec to actual state. The architectural shape is similar to Cluster API, scoped to bare metal rather than VMs.

I am working on the `wipe-disk` command — secure-erase of all drives on a host before reprovisioning, using NVMe Sanitize where the controller advertises it and falling back to `hdparm --security-erase`. The CLI orchestrates: power-on via IPMI, MAAS commission into an ephemeral OS, push the wipe script over SSH, run it, wait for `poweroff -f`, MAAS releases the machine. End-to-end across a real bare-metal pipeline.

2.3 Independent project: PostgreSQL on Kubernetes / OpenStack

A 5-layer nested virtualization stack on a single i9-14900 box: bare-metal Linux at the bottom, KVM running an OpenStack tenancy on top, Kubernetes running inside that, CloudNativePG inside Kubernetes, Postgres inside CNPG. The work was equal parts performance tuning, lifecycle automation, and identity plumbing.

Performance tuning. The inner-VM Postgres saw write-TPS climb roughly 50% after two pieces of work — NUMA-aware CPU pinning of the Postgres workers to the same socket as the L1 hypervisor exposing the disks, and 16 GB of hugepages dedicated to the inner VM. The buffer pool was thrashing the TLB at default page sizes; 2 MB pages cut TLB misses enough that the syscall cost on every page fault went away.

Lifecycle automation. Cluster API for OpenStack (CAPO) provisions VMs declaratively; CloudNativePG is the Kubernetes operator that runs the Postgres cluster. End-to-end, a single `kubectl apply` brings up an HA Postgres cluster (one primary + N replicas, synchronous WAL streaming) in ~12 minutes from cold. Failover is automatic — CNPG runs a leader-election loop, demotes the failing primary, promotes a healthy replica, and rewires the service endpoint. Tested under primary-VM kill and primary-VM network partition; recovery to write-ready in roughly 7 seconds with RPO=0 (no acknowledged write lost, because the synchronous stream guarantees the standby has it before the primary acks the client).

Identity plumbing. A Go-based OpenBao broker plus the `osvault` Python SDK on the VM side delivers secrets at bootstrap — the VM proves its OpenStack-assigned identity via IMDS, the broker validates it, the SDK fetches the secrets it is allowed to.

2.4 urunc so far — what I have learned through issue #389

A few weeks ago I picked up [urunc-dev/urunc#389](#) — high host CPU caused by console output in running containers. A prior contributor had attributed the regression to `bufio.Scanner` in `cmd/urunc/log_forward.go`. My own profiling showed that path is not hot during guest execution: that scanner only handles the parent-child JSON log pipe during `urunc create`; once `syscall.Exec()` runs the VMM manages stdio directly. The real cost lives one layer deeper.

I wrote a small KVM-exit and throughput sampler — `bench-console.sh`, which @ananos has since indicated they would like to fold into CI as a regression check — and traced the regression to per-byte PIO traps on the 16550 UART at port `0x3F8`. The `-serial stdio` baseline costs roughly 1.16 M PIO exits per 5-second window, with QEMU's `address_space_write` dominating the host flame graph. A simple `virtio-console` swap drops PIO exits to ~0 and lifts throughput from 2,016 to 73,378 lines/sec (about 36×), closing the runc-vs-urunc gap on this workload from roughly 407× down to ~11×.

The thread has since covered: how to differentiate `virtio-console` from `serial` without breaking Linux kernels built without `CONFIG_VIRTIO_CONSOLE` (my first proposal, a capability-intersection design borrowed from the rootfs path, was constructively rejected by @cmainas for exactly that reason); the upcoming pivot from the `com.urunc.unikernel.cmdline` annotation to a `kernelArgs`-style annotation that lets users specify boot parameters at build time; and an invitation from @ananos to draft a blog post on the investigation for the CNCF blog.

Through that work I have gained working familiarity with:

- Monitor invocation path — `pkg/unikontainers/hypervisors/qemu.go`, `BuildExecCmd`, and the `syscall.Exec` -style handoff in `unikontainers.go`.
- The distinction between the parent-child JSON log pipe (consumed by `bufio.Scanner` before `syscall.Exec`) and the VMM-managed stdio path that takes over after.
- The QEMU + KVM monitor surface — what costs PIO exits at the 16550 UART, how virtio devices change the picture, where the cost actually lives.
- Linux unikernel path — `pkg/unikontainers/unikernels/linux.go`, the boot-params construction, the urunit handshake.
- The `VMM` interface in `pkg/unikontainers/types/types.go` and how each monitor (QEMU, Firecracker, HVT) implements it today.
- An empirical debugging posture — `/sys/kernel/debug/kvm/io_exits`, `perf` flame graphs, and a small reproducer image, rather than guessing.

3. The project — monitor lifecycle for urunc#112

3.1 The problem

urunc today shells out to QEMU or Firecracker with a long command string. That is enough to *start* a sandbox, but everything after that — checking what the guest is doing, hotplugging a device, asking the monitor to shut down gracefully, attaching a console after the fact — is either impossible or has to be approximated by signalling the host process. Once `syscall.Exec` hands off to the monitor at `unikontainers.go:533`, the only handle urunc retains is the PID. Both QEMU and Firecracker already expose proper socket APIs — QMP for QEMU, a REST-style API socket for Firecracker — and the [slide deck for #112](#) calls out a secondary benefit: the `reexec` process currently blocks on `AwaitMsg(StartExecve)` between the create and start phases, and that gap can be used for parallel VMM init for a ~30 ms boot win.

3.2 Verifying the wire protocols first

Before writing the proposal I built two small Go clients to drive each monitor through its API end-to-end. Both live in this repo under `prototypes/`, with run scripts and full transcripts.

`prototypes/qmp-probe/` drives a real `qemu-system-x86_64 8.2.2` through its QMP Unix socket — greeting, `qmp_capabilities`, `query-status` before and after `cont`, `query-cpus-fast`, `stop`, `quit`. Full transcript in ~2 ms wall time. The non-obvious find: QMP interleaves async events with command replies on the same socket. My first version of the client was naive — one read per command — and worked through the handshake, then `cont`'s `RESUME` event arrived before the `cont` reply and every subsequent read got off-by-one. The fix is to type-dispatch frames (`{return}` / `{error}` are replies; `{event, timestamp, data}` is telemetry), but it is the kind of thing that does not surface from reading docs.

`prototypes/firecracker-probe/` drives Firecracker v1.13.1 through its API socket — `GET /`, `PUT /machine-config`, `PUT /boot-source`, `PUT /drives/rootfs`, `PUT /actions {InstanceStart}`, `PATCH /vm {Paused/Resumed}`, `PUT /actions {SendCtrlAltDel}`. Booted a real Ubuntu 22.04

microVM end-to-end; guest reached systemd, honored Ctrl-Alt-Del, Firecracker exited with `exit_code=0`. `InstanceStart` was the only expensive call at 16 ms; everything else was 0.3–0.5 ms. Three contrasts with QMP that matter for the interface: Firecracker has no CLI for guest config (API-only), no async events on the socket, and no `device_add` equivalent (you can `PATCH` existing drives but not attach new devices after boot).

3.3 My approach

The change has two pieces that have to land together: a monitor-agnostic control interface inside urunc, and per-monitor implementations against it. The interface lives next to the existing `VMM` interface in `pkg/unikontainers/hypervisors/types.go` and covers the operations urunc actually needs — probe state, pause, resume, shutdown, hotplug net/block, attach console. Backends carry capability flags for asymmetries (Firecracker has no events; HVT has no socket at all and falls back to the existing CLI-only path).

The architectural choice I would make: preserve urunc's existing OCI semantic that the container's init process *is* the monitor itself. No supervisor process between the runtime and the container; `state.json::containerPid` stays the monitor PID. The flow change:

- The `reexec` process, after `CreateContainer` hooks but before blocking on `AwaitMsg(StartExecve)`, spawns the monitor in background mode as a child via `fork+exec` (not `execve`). For QEMU that is `-S -qmp unix:<path>,server,nowait`; for Firecracker it is `--api-sock <path>`. The monitor is now its own process; the socket is open.
- While `reexec` waits for `StartExecve`, the monitor initialises devices in parallel — this is the ~30 ms boot win the slide deck targets, and the Firecracker probe confirms it (every config `PUT` is sub-millisecond; only `InstanceStart` costs 16 ms).
- When `StartExecve` arrives, `reexec` connects to the monitor's socket and sends the "start guest" command — `cont` for QEMU, `PUT /actions {InstanceStart}` for Firecracker. It sends `StartSuccess` back to urunc start, then exits.
- For lifecycle ops after start (`urunc kill`, `urunc delete`, a future `urunc monitor query-status`), each command opens a fresh connection to the persisted socket, sends the operation, closes. Per the QMP probe, that is sub-millisecond. Fallback for dead sockets is signal-to-PID, as today.

I prefer this over a long-running supervisor for three reasons. First, it preserves the OCI semantic the shim already depends on — the container's init process is the monitor, not a wrapping process. Second, the failure modes are simpler: a dead monitor means a dead PID and a dead socket, with no live-supervisor-but-dead-monitor zombie state. Third, the persistent-connection benefit of a supervisor is small in practice — Unix-socket reconnect is sub-millisecond, and async-event subscription (the only thing that genuinely benefits from a long-lived connection) can be a separate event-sink process without becoming the container's init.

3.4 Concrete changes in code

- `pkg/unikontainers/hypervisors/qemu.go` — extend `BuildExecCmd` with `-qmp unix:<sock>,server,nowait -S`. Implement the new control interface for QMP, prioritising `query-`

`status` (smallest, exercises every path), `quit` (replaces `killProcess`), then `cont / stop`, `device_add / device_del`, `chardev-add / chardev-remove`.

- `pkg/unikontainers/hypervisors/firecracker.go` — implement the same interface backed by `firecracker-go-sdk`. Move from config-file-then-boot to incremental `PUT` calls during the create/start gap; only `PUT /actions {InstanceStart}` when `StartExecve` lands.
- `pkg/unikontainers/hypervisors/types.go` — add the control interface (working name `MonitorControl`) alongside `VMM`. Sibling rather than extension because the lifetimes differ: `VMM` methods are valid from spawn; `MonitorControl` methods are valid only after the socket handshake.
- `pkg/unikontainers/unikontainers.go` — the load-bearing change. `Exec` (currently ending at `syscall.Exec` on line 533) splits into spawn-monitor-in-background, wait-for-StartExecve, send-start-via-socket, send-StartSuccess. Reexec exits cleanly after sending start; it no longer execs into the monitor.
- `cmd/urunc/kill.go`, `cmd/urunc/delete.go` — try `MonitorControl.Shutdown()` first (QMP `quit`, FC `SendCtrlAltDel`), fall back to signal-to-PID if the socket is broken or unresponsive.
- `pkg/unikontainers/ipc.go` — likely a small addition for a `MonitorReady` state between background-spawn and `StartExecve`. The existing message-with-retry helpers in this file already handle the "socket not ready yet" race for urunc's own sockets and translate cleanly.

3.5 Out of scope and what I will verify

Out of scope for this LFX even if time allows: rewriting urunc's seccomp handling (`PreExec` for HVT), adding a third monitor (cloud-hypervisor, Dragonball — the interface should make it easier, but doing it is somebody else's first PR), introducing a heavy QMP or FC client library when the prototypes already show both wires are small enough to hand-roll, and rewriting the existing reexec/urunc-start IPC channels.

Before each merge I commit to verifying: the QMP-controlled QEMU path can spawn paused, query state, gracefully shut down, and hotplug a virtio-net device end-to-end with the existing nginx-on-unikraft integration tests staying green; the Firecracker path can do the equivalent subset; the CLI-only fallback continues to work for HVT and any monitor that does not expose a socket; an extended `bench-console.sh` measures per-round-trip cost on both backends and lands in CI as a regression check.

4. Plan — 12 weeks (8 Jun – 31 Aug 2026)

The design doc opens as a draft PR from week 1 and stays open through week 3 for asynchronous comments. Implementation is broken into multiple small PRs so review happens in parallel with development. The verification work in §3.2 reduces risk on weeks 4–7 — I am not learning QMP or the Firecracker API for the first time during the implementation.

WK	PHASE	DELIVERABLE
1–2	Onboarding + design doc draft	Re-read <code>pkg/unikontainers/hypervisors/</code> with mentor context. Open a draft design-doc PR covering the <code>MonitorControl</code> interface, the

WK	PHASE	DELIVERABLE
		QMP and Firecracker mapping, the reexec flow change, socket lifecycle, and the opt-in vs default-on policy.
3	Design doc sign-off	Address review on the draft PR. Interface signature and dependency decisions settled before any production code lands.
4–5	QEMU + QMP integration	Extend the qmp-probe code into urunc's QEMU backend. MVP first (<code>query-status</code> + <code>quit</code> + <code>cont</code>), then full lifecycle (<code>stop</code> / <code>cont</code> , <code>device_add</code> / <code>device_del</code> for virtio-net and virtio-blk). Unit tests for the wire codec.
6–7	Firecracker + reexec rewiring	Same <code>MonitorControl</code> surface for FC backed by <code>firecracker-go-sdk</code> . Land the reexec flow change: monitor spawned in background mode during the create/start gap, legacy <code>syscall.Exec</code> path retained behind a feature flag.
8–9	Tests, CI, hotplug demo	Integration tests in the existing layout. Extend <code>bench-console.sh</code> to time the new round-trip costs. End-to-end demo: start a unikraft-nginx sandbox, hotplug a second virtio-net device, verify connectivity inside the guest.
10	Docs + bug-fix	Review-comment cleanup across all open PRs. User docs covering the QMP socket and how to opt into the new control path.
11–12	Buffer + blog post	Reserved for slippage. If none, take a follow-up issue from the maintainers' backlog. Mentee blog post for the CNCF / Nubificus blog covering the redesign and the numbers.

The riskiest piece is the reexec flow change — the IPC sequencing between reexec, the background monitor, and urunc start. Mitigation: feature-flag it from week 4; the legacy `syscall.Exec` path stays default until the new path has green integration tests in week 8.